

При последовательном доступе к данным возможно выполнение только последовательного чтения элементов данных или последовательная их запись. Такой вариант предполагается при работе с логическими устройствами типа клавиатуры или дисплея, при обработке текстовых файлов или файлов, формат записей которых меняется в процессе работы.

Прямой доступ возможен только для дисковых файлов, обмен информацией с которыми осуществляется записями фиксированной длины (двоичные файлы C или типизированные файлы Pascal). Адрес записи такого файла можно определить по ее *номеру*, что и позволяет напрямую обращаться к нужной записи.

При выборе типа памяти для размещения структур данных следует иметь в виду, что:

- в оперативной памяти размещают данные, к которым необходим быстрый доступ как для чтения, так и для их изменения;
- во внешней - данные, которые должны сохраняться после завершения программы.

Возможно, что во время работы данные целесообразно хранить в оперативной памяти для ускорения доступа к ним, а при ее завершении - переписывать во внешнюю память для длительного хранения. Именно этот способ используют большинство текстовых редакторов: во время работы с текстом он весь или его часть размещается в оперативной памяти, откуда по мере необходимости переписывается во внешнюю память. В подобных случаях разрабатывают два представления данных: в оперативной и во внешней памяти.

Правильный выбор структур во многом определяет эффективность разрабатываемого программного обеспечения и его технологические качества, поэтому данному вопросу должно уделяться достаточное внимание независимо от используемого подхода.

5.5. Проектирование программного обеспечения, основанное на декомпозиции данных

В § 4.5 уже упоминалось, что практически одновременно были предложены методики проектирования программного обеспечения Джексона и Варнье-Орра, основанные на декомпозиции данных. Обе методики предназначены для создания «простых» программ, работающих со сложными, но иерархически организованными структурами данных. При необходимости разработки программных систем в обоих случаях предлагается вначале разбить систему на отдельные программы, а затем использовать данные методики.

Методика Джексона. При создании своей методики М. Джексон исходил из того, что структуры исходных данных и результатов определяют структуру программы.

Методика основана на поиске соответствий структур исходных данных и результатов. Однако при ее применении возможны ситуации, когда на каких-то уровнях соответствия отсутствуют. Например, записи исходного файла сортированы не в том порядке, в котором соответствующие строки должны появляться в отчете. Такие ситуации были названы «столкновениями». Выделяют несколько типов столкновений, которые разрешают по-разному. При различной последовательности записей их просто сортируют до обработки. Более подробно способы разрешения столкновений изложены в [33].

Разработка структуры программы в соответствии с методикой выполняется следующим образом:

- строят изображение структур входных и выходных данных;
- выполняют идентификацию связей обработки (соответствия) между этими данными;
- формируют структуру программы на основании структур данных и обнаруженных соответствий;
- добавляют блоки обработки элементов, для которых не обнаружены соответствия;
- анализируют и обрабатывают несоответствия, т.е. разрешают «столкновения»;
- добавляют необходимые операции (ввод, вывод, открытие/закрытие файлов и т. п.);
- записывают программу в структурной нотации (псевдокоде).

Пример 5.4. Разработать структуру программы, которая читает записи об успеваемости студентов и формирует список неуспевающих студентов группы.

На рис. 5.17 представлены структуры входных и выходных данных программы. Анализ этих структур показывает, что между ними есть соответствия (на рис. 5.17 эти соответствия показаны полужирными дугами). Помимо полного соответствия, имеет место еще частичное соответствие - соответствие, отмечаемое только, если студент имеет задолженности (на рис. 5.17 оно отмечено полужирным пунктиром).

Используя найденные полные и неполные соответствия, строим «каркас» программы (затемненные блоки на рис. 5.18). Согласно методике добавляем блоки, которые позволят разрешить «столкновения» (светлые блоки на рис. 5.18).

Далее строим полный список операций, которые должна выполнять программа, учитывая, что не каждой записи исходного файла соответствует строка отчета (признак «формировать запись вывода» установлен), и выводить надо названия только тех предметов, по которым у студента есть задолженности (признак «задолженность» установлен):

- 1 - завершить работу;
- 2 - открыть входной файл;
- 3 - открыть выходной файл;

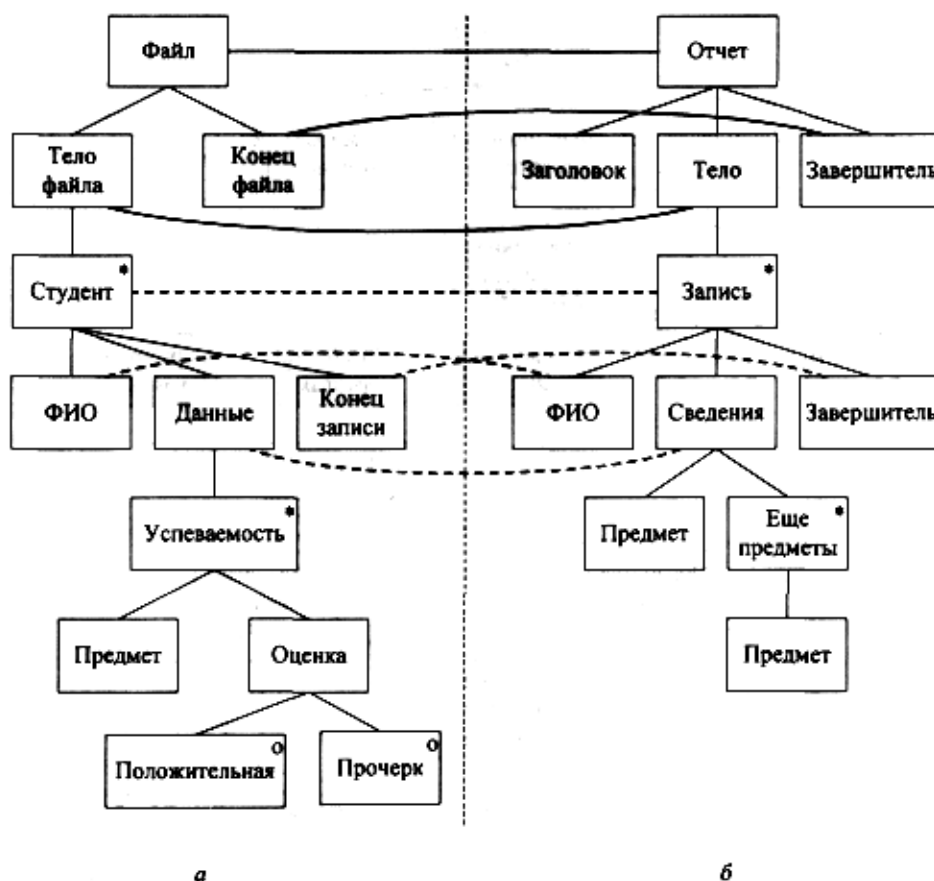


Рис. 5.17. Структуры входных (а) и выходных (б) данных программы

- 4 - закрыть входной файл;
- 5 - закрыть выходной файл;
- 6 - вывести заголовок;
- 7 - вывести завершитель;
- 8 - ввести запись входного файла;
- 9 - вывести строку отчета (при включенном состоянии признака «формировать запись вывода»);
- 10 - очистить буфер вывода;
- 11 - установить признак «формировать запись вывода»;
- 12 - сбросить признак «формировать запись вывода»;
- 13 - поместить в строку вывода ФИО;
- 14 - установить признак «задолженность»;
- 15 - сбросить признак «задолженность»;

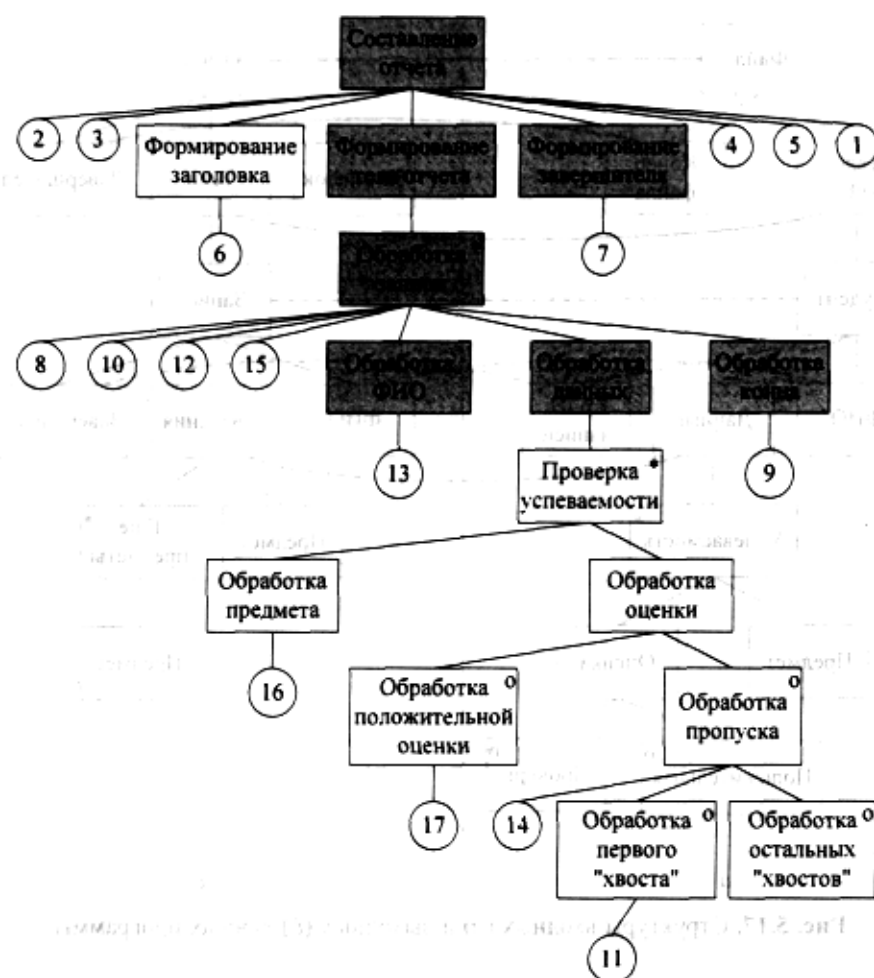


Рис. 5.18. Структура программы формирования списка студентов с задолженностями:

- – блоки, полученные при анализе соответствий;
- – блоки, полученные при анализе несоответствий;
- – блоки, добавленные при анализе операций

16- занести название предмета в строку вывода;

17- стереть название предмета из буфера.

Затем определяем местоположение операций обработки (на рис. 5.18 они показаны кружочками с номерами). Результатом является полная структура разрабатываемой программы в нотации Джексона. Далее в соответствии с методикой следует записать алгоритм программы на псевдокоде.

В методике Джексона предлагается псевдокод, точно соответствующий графической нотации. Он использует следующие конструкции.

Последовательность:

<Имя> Посл.

Выполнить <действие 1>

Выполнить <действие 2> <Имя>

КОНЕЦ

Выбор:

*<Имя> **Выбор** <условие действия 1>*

Выполнить <действие 1> <Имя>

или $\langle \text{условие действия } 2 \rangle$

Выполнить <действие 2> <Имя>

КОНЕЦ

Повторение.

<Имя> **Повт**, пока не <условие действия>

Выполнить <действие 1> <Имя>

КОНЕЦ

С применением этого псевдокода запись алгоритма программы выглядит следующим образом:

Составление отчета посл.

Открыть входной файл

Открыть выходной файл Вывести заголовок отчета

Формирование тела отчета **повт.** пока не конец входного файла.

Ввести запись

Очистить буфер вывода

Сбросить признак «формировать запись вывода»

Сбросить признак «задолженность»

Вывести в буфер ФИО

Обработка данных **повт**, пока не конец записи

*Обработка предмета **посл.***

Занести название предмета в строку вывода

*Обработка предмета **конец***

Обработка оценки **выбор** если оценка положительна

Стереть название предмета из буфера

Обработка оценки **или** если оценка пропуск Установить признак «задолженность» Обработка пропуска **выбор** если не установлен признак «формировать запись вывода»

Установить признак «формировать запись вывода»
 Обработка пропуска **конец**
 Обработка оценки **конец**
 Обработка конца записи **выбор** если установлен признак «формировать запись вывода»
 Вывести строку отчета
 Обработка конца записи **конец**
 Обработка данных **конец**
 Формирование тела отчета **конец**
 Вывести завершитель
 Закрывать входной файл
 Закрывать выходной файл
 Завершить работу
 Составление отчета **конец**

Методика Варнье-Орра. Методика Варнье-Орра базируется на том же положении, что и методика Джексона, но основными при построении программы считаются структуры выходных данных и, если структуры входных данных не соответствуют структурам выходных, то их допускается менять. Таким образом, ликвидируется основная причина столкновений. В примере 5.4 целесообразно поменять местами оценки и названия предметов, чтобы упростить обработку.

Однако на практике не всегда существует возможность пересмотра структур входных данных: эти структуры уже могут быть строго заданы, например, если используются данные, полученные при выполнении других программ, поэтому данную методику применяют реже.

Как следует из вышеизложенного, методики Джексона и Варнье-Орра могут использоваться только в том случае, если данные разрабатываемых программ могут быть представлены в виде иерархии или совокупности иерархий.

5.6. Case-технологии, основанные на структурных методологиях анализа и проектирования

К нашему времени накоплен опыт успешного использования большинства известных методологий структурного анализа и проектирования в соответствующих CASE-средствах. Наибольшее распространение получили методологии [30]: SADT (3,3%), структурного системного анализа Гейна-Сарсона (20,2%), структурного анализа и проектирования Йордана-Де Марко (36,5%), развития систем Джексона (7,7%), развития структурных схем DSSD (Data Structured System Development), Варнье-Орра (5,8%), анализа и проектирования систем реального времени Уорда-Меллора и Хатли, информационного моделирования Мартина (22,1%).